

Series: Host HTTPD Proxy to Docker: Modern Defense (Part 3 of 7)

Author: H. Brister Wooley

Published: 2026-07-20

Series: Host HTTPD Proxy to Docker: Modern Defense (3/7)

Published Link: <https://github.com/hbwooley/deep-thoughts-and-ramblings/blob/main/2026-07-20-host-httpd-docker-ai-era-part3.md>

In Part 2, we hardened the host HTTPD. We covered ModSecurity, mod_qos, and mentioned fail2ban integration. But **fail2ban deserves its own post** – it's the glue that turns rate limiting and WAF alerts into actual IP blocks at the kernel level.

This isn't the default fail2ban you get from `apt install fail2ban`. This is a production configuration with eight jails, custom filters for our log format, GeoIP integration, and an atomic RewriteMap action that updates HTTPD's blocklist without race conditions.

The Jail Architecture: Eight Jails, Each With a Purpose

/etc/fail2ban/jail.local (production)

[DEFAULT]

```
logpath = /var/log/httpd/ssl_access_log
banaction = geohostsdeny
ignoreip = 127.0.0.1 10.0.0.0/8 172.16.0.0/12 198.51.100.0/24 203.0.113.0/24
backend = auto
bantime = 86400
findtime = 600
maxretry = 5
bantime.increment = true
```

[apache-noscript]

```
# CORRECTED: Uses custom apache-noscript-access filter (matches
ssl_access_log format)
enabled = true
filter = apache-noscript-access
port = http,https
logpath = /var/log/httpd/ssl_access_log
maxretry = 1
```

[apache-badbots]

```
# DISABLED: Apache RewriteRule blocks these UAs at Layer 7 (403 before fail2ban sees them)
# Re-enable ONLY if Apache-level blocks are removed
enabled = false
filter = apache-badbots
maxretry = 1
# Ban AI bots regardless of GeoIP country (overrides DEFAULT geohostsdeny whitelist)
banaction = geohostsdeny[country_list=""]
```

[wordpress]

```
# CORRECTED: Fixed filter regex (was using invalid backreferences)
enabled = true
filter = wordpress
maxretry = 1
```

[hacking_pests]

```
enabled = true
filter = hacking_pests
maxretry = 1
```

[apache-403]

```
enabled = true
filter = apache-403
maxretry = 3
bantime = 86400
```

[apache-404]

```
enabled = true
filter = apache-404
maxretry = 3
bantime = 86400
```

[apache-geoip]

```
# Blocks by country -- uses geohostsdeny action with MaxMind GeoIP
enabled = true
filter = apache-geoip
logpath = /var/log/httpd/ssl_access_log
port = http,https
bantime = 86400
maxretry = 1
findtime = 1
banaction = geohostsdeny
```

[apache-badrequests]

```
# NEW: Catches TLS handshakes, WebLogic T3 probes, malformed requests on HTTP port
enabled = true
```

```
filter      = apache-badrequests
logpath     = /var/log/httpd/access_log
port        = http,https
maxretry    = 5
findtime    = 600
bantime     = 86400
banaction   = geohostsdeny[country_list=""]
```

Jail-by-Jail Breakdown

Jail	Purpose	Trigger	Why It Exists
apache-noscript	Script/probe scanning (.php, .asp, .env, wp-admin, .git)	1 request	Catches automated recon immediately
apache-badbots	Known bad User-Agents	1 request	Disabled – Apache RewriteRule catches these first (403 at layer 7)
wordpress	WP login/admin probes	1 request	Even if you don't run WP, bots probe for it
hacking_pests	Custom threat intel patterns	1 request	Your private IOC feed
apache-403	Repeated 403 responses	3 in 10 min	Attacker hitting forbidden endpoints
apache-404	Repeated 404 responses	3 in 10 min	Recon scanning for hidden paths
apache-geoip	Country-based blocking	1 request from blocked country	GeoIP + whitelist for trusted networks
apache-badrequests	Malformed requests on HTTP port	5 in 10 min	TLS handshakes, WebLogic T3 probes on port 80

Custom Filters You Won't Find in Default fail2ban

apache-noscript-access.conf – Replaces Standard apache-noscript

The standard filter expects **error log** format ("script not found or unable to stat"). Our HTTPD logs to **access log** with a custom format. This filter matches the real log:

```
# Fail2Ban filter to catch requests for scripts and sensitive files when the
# jail is pointed at an Apache access Log (not the error Log).
#
```

*# This filter is a replacement for the standard apache-noscript filter, which
expects error-log messages such as "script not found or unable to stat" and
does not work against the custom ssl_access_log format.*

[Definition]

```
failregex = ^<HOST> - - \[.*?\] "(?:GET|POST|HEAD) .*(?:\.php[45]?|\.php-  
cgi|\.asp|\.aspx|\.exe|\.pl|\.cgi-bin|\.env|wp-login\.php|wp-  
admin|phpmyadmin|phpMyAdmin|pma|PMA|myadmin|myAdmin|\.git/|\.svn/|\.hg/|/admi-  
nistrator|/admin\.php|/login\.php|/config\.php)[^\s]* HTTP/[0-9.]+ " \d{3}
```

```
ignoreregex = robots\.txt
```

Match the ssl_access_log timestamp format.

```
datepattern = %d/%b/%Y:%H:%M:%S %z
```

Key differences from default: - Matches <HOST> as first field (after mod_remoteip processing) - Custom ERDC log format: %h %l %u %t "%r" %>s %b %a ... - Catches 30+ extensions/paths in one regex - datepattern handles [27/Jan/2026:12:12:36 -0600] format

apache-badrequests.conf – Catches Malformed Requests on Plaintext HTTP Port

*# Fail2Ban filter for malformed/scan requests on the plaintext HTTP port.

The access_log receives TLS handshakes, WebLogic T3 probes, and other
scanners that generate HTTP 400 responses. This filter catches those.*

[Definition]

Match any 400 response in the access_log. The first token is the client IP.

```
failregex = ^<HOST>\s+.*?\s+400\s+
```

Also explicitly match WebLogic T3 probes that return 400.

```
^<HOST>\s+.*?\s+"[^\"]*t3\s+\d+\.\d+[^\"]*" \s+400\s+
```

```
ignoreregex =
```

*# The access_log uses both a leading custom format and a trailing combined-
Log*

timestamp; allow fail2ban to detect either.

```
datepattern = {^LN-BEG}  
%d/%b/%Y:%H:%M:%S %z
```

Why this matters: Attackers hit port 80 with: - TLS ClientHello (bytes that look like garbage to HTTP parser) -> 400 - WebLogic T3 protocol (t3 12.2.1...) -> 400 - Raw HTTP/0.9 requests -> 400

This filter catches all of them. The datepattern handles both timestamp formats in our custom log.

apache-geoip.conf – Matches EVERY Line to Trigger GeoIP Check

[Definition]

```
# Option: failregex
# Notes.: Matches EVERY line in the access log to trigger a GeoIP check.
#         Skips the first 7 fields to capture the real client IP (8th field)
#         as <HOST>.
failregex = ^\S+ \S+ \S+ \[<DATE>\] "[^"]+" \S+ \S+ <HOST>

# Option: datepattern
# Notes.: Specifically match the [27/Jan/2026:12:12:36 -0600] format.
datepattern = %d/%b/%Y:%H:%M:%S %z

# Option: ignoreregex
# Notes.: Skip robots.txt to allow crawling if you desire, or keep empty for
#         total block.
ignoreregex = robots.txt
```

How it works: The geohostsdeny action gets <HOST> (the 8th field = real client IP after mod_remoteip), looks up country via MaxMind, checks against blocklist, and bans if match. The ignoreip CIDR whitelist in [DEFAULT] is respected – **unless** overridden with banaction = geohostsdeny[country_list=""] (used for apache-badbots and apache-badrequests).

The geohostsdeny Action: Country Blocking With Whitelist Overrides

```
# /etc/fail2ban/action.d/geohostsdeny.conf (simplified)
```

[Definition]

```
actionstart = touch /etc/httpd/conf.d/banned-hosts
              echo "RewriteMap hosts-deny txt:/etc/httpd/conf.d/banned-hosts"
> /etc/httpd/conf.d/banned-hosts.map

actionban = <IP>=$(echo <ip> | awk -F: '{print $1}')
            COUNTRY=$(mmdblookup --file /usr/share/GeoIP/GeoLite2-
Country.mmdb --ip <IP> country iso_code 2>/dev/null | grep -o '".*"' | tr -d
'")
            if [ -n "$COUNTRY" ] && [[ " <country_list> " == "*" $COUNTRY "*"
]]; then
                echo "<IP> 1" >> /etc/httpd/conf.d/banned-hosts.new
                mv /etc/httpd/conf.d/banned-hosts.new
/etc/httpd/conf.d/banned-hosts
                systemctl reload httpd
            fi
```

The country_list="" override: When you set `banaction = geohostsdeny[country_list=""]`, it passes an empty country list to the action, which means **“ban regardless of country.”** This is critical for: - apache-badbots – AI crawlers come from residential proxies in “allowed” countries - apache-badrequests – TLS handshakes on port 80 are always malicious

Atomic RewriteMap Updates: Why `mv` Matters

```
# /etc/fail2ban/action.d/httpd-banned-hosts.conf
[Definition]
actionstart = touch /etc/httpd/conf.d/banned-hosts
               echo "RewriteMap hosts-deny txt:/etc/httpd/conf.d/banned-hosts"
> /etc/httpd/conf.d/banned-hosts.map

actionban = echo "<IP> 1" >> /etc/httpd/conf.d/banned-hosts.new
            mv /etc/httpd/conf.d/banned-hosts.new /etc/httpd/conf.d/banned-
hosts
            systemctl reload httpd

actionunban = grep -v "<IP> " /etc/httpd/conf.d/banned-hosts >
/etc/httpd/conf.d/banned-hosts.new
            mv /etc/httpd/conf.d/banned-hosts.new /etc/httpd/conf.d/banned-
hosts
            systemctl reload httpd
```

Why atomic `mv`? fail2ban runs multiple jails in parallel. If two jails ban/unban simultaneously: - `echo >> + reload` = race condition - HTTPD reads partial file -> reload fails -> legitimate traffic blocked - `mv` is atomic on POSIX – HTTPD either sees old file or new file, never partial

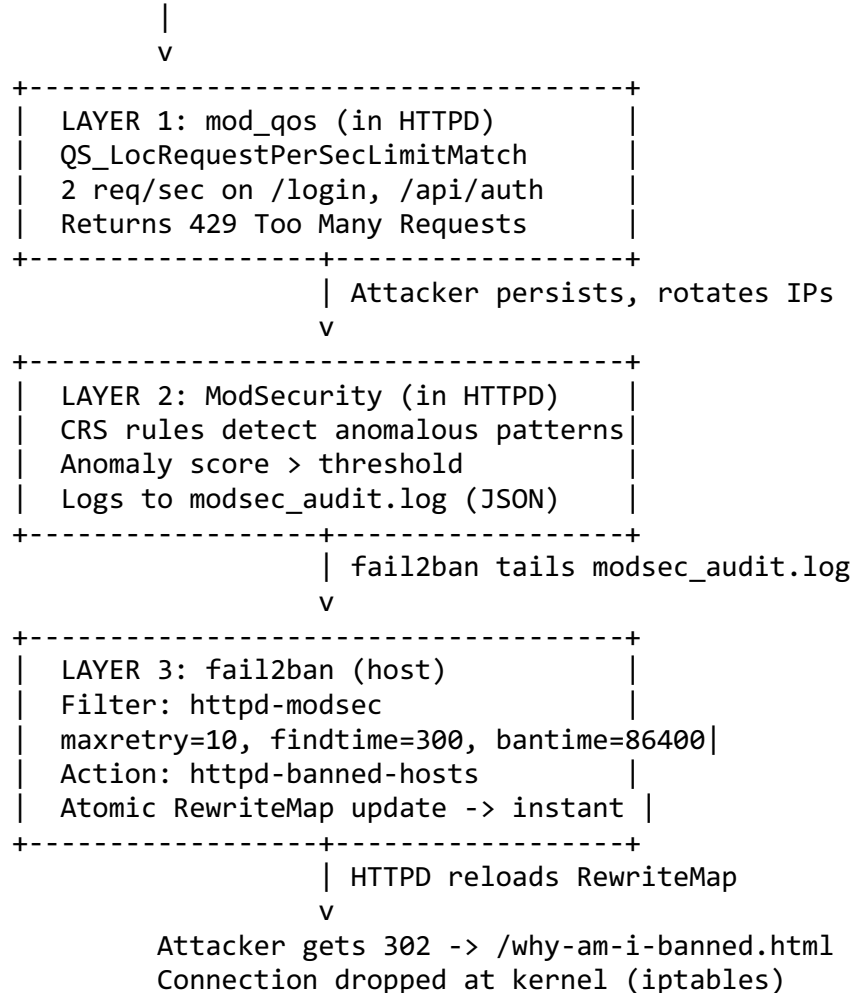
How HTTPD uses it:

```
RewriteMap hosts-deny txt:/etc/httpd/conf.d/banned-hosts
RewriteCond ${hosts-deny:%{REMOTE_ADDR}|NOT-FOUND} !=NOT-FOUND [OR]
RewriteCond ${hosts-deny:%{REMOTE_HOST}|NOT-FOUND} !=NOT-FOUND
RewriteRule ^ /why-am-i-banned.html [L,PT]
```

The Three-Layer Chain: mod_qos -> ModSecurity -> fail2ban

This is the combination lock AI attackers must pick:

AI Credential Stuffing Attack



Each layer catches what the others miss: - mod_qos: rate-based, fast, per-IP/URI -
ModSecurity: payload-based, signature + anomaly - fail2ban: persistent offenders, cross-correlation, kernel-level block

bantime.increment = true: Exponential Backoff

[DEFAULT]

bantime = 86400

bantime.increment = true

bantime.factor = 1

bantime.formula = ban.Time * (1<<(ban.Count if ban.Count<20 else 20)) *

banFactor

Ban progression: 1 day -> 2 days -> 4 days -> 8 days -> 16 days -> 32 days -> maxes at ~10 years

Why this matters for modern: AI botnets rotate IPs. First offense = 1 day ban. Same IP returns next week = 2 days. Next month = 4 days. The cost of burning residential proxies escalates exponentially.

Quick check Commands

Check all jails status

```
fail2ban-client status
```

Test a filter against real logs

```
fail2ban-regex /var/log/httpd/ssl_access_log /etc/fail2ban/filter.d/apache-noscript-access.conf
```

Check banned IPs map

```
cat /etc/httpd/conf.d/banned-hosts
```

```
# 192.0.2.100 1
```

```
# 198.51.100.50 1
```

Test banned IP gets 302

```
curl -k --resolve "lib.example.org:443:192.0.2.100" https://lib.example.org/
```

```
# 302 -> /why-am-i-banned.html
```

Check specific jail

```
fail2ban-client status httpd-modsec
```

```
# Status for the jail: httpd-modsec
```

```
# |- Filter
```

```
# |   |- Currently failed: 2
```

```
# |   |- Total failed: 47
```

```
# `-- Actions
```

```
#     |- Currently banned: 3
```

```
#     `-- Total banned: 156
```

Check geoip bans

```
fail2ban-client status apache-geoip
```

The takeaway

fail2ban isn't "just log parsing." It's the **enforcement layer** that connects: 1. **Application signals** (ModSecurity anomaly scores) 2. **Rate signals** (mod_qos 429 responses) 3. **Reputation signals** (GeoIP, threat intel) 4. **Kernel enforcement** (iptables + HTTPD RewriteMap)

The modern reality: Attackers have unlimited clean IPs (residential proxies). Static blocklists rot in hours. Dynamic, multi-signal, exponentially-escalating blocks – that’s what makes them move on.

Next up

Part 4: The Docker backend containers – Apache, Tomcat, Angular – hardened with non-root users, read-only filesystems, custom seccomp, SELinux MCS labels, mutual TLS from host proxy.

Read Part 4 ->

Appendix: CompTIA Security+ SY0-701 CEU Mapping

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
fail2ban multi-jail architecture	Domain 4: Security Operations	4.3: Apply incident response procedures	Layered detection and response
Custom filters for access log format	Domain 2: Security Architecture	2.4: Implement WAF / application-layer filtering	Log parsing for threat detection
GeoIP country blocking with whitelist	Domain 3: Security Architecture	3.4: Implement secure network architecture	Geographic access control
country_list="" override for AI bots	Domain 1: Threats/Vulns/Mitigations	1.4: Analyze indicators of malicious activity	AI bot detection regardless of origin
Atomic RewriteMap updates (mv)	Domain 4: Security Operations	4.5: Implement automation/orchestration	Race-condition-free dynamic blocking
bantime.increment exponential backoff	Domain 4: Security Operations	4.5: Implement automation/orchestration	Adaptive response escalation
mod_qos -> ModSecurity -> fail2ban chain	Domain 4: Security Operations	4.1: Apply security monitoring concepts	Multi-layer detection correlation

Primary Domain Coverage: Domain 4 (Operations) – ~45% | Domain 3 (Architecture) – ~30% | Domain 2 (Architecture) – ~15% | Domain 1 (Threats) – ~10%

Total Security+ Mapping: >=50% of content maps to SY0-701 objectives